

Reinforcement Learning

Clase 08

Ignacio Brevis

`ignaciobrve@unisabana.edu.co`

Universidad de La Sabana

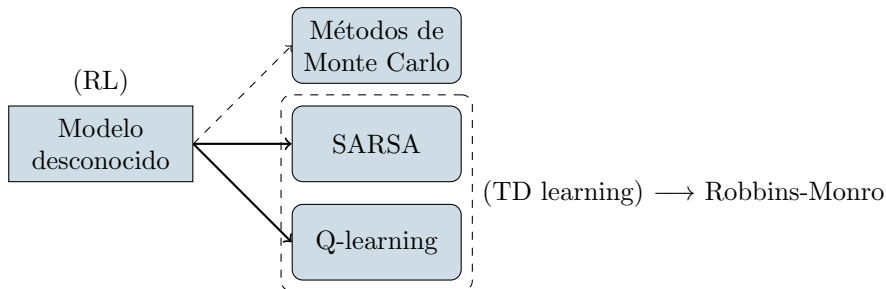
07 octubre 2026



Repaso: TD methods

Los métodos de **temporal-difference** (TD) learning combinan ideas de los métodos de Monte Carlo y de programación dinámica (DP).

- **MC: No requiere modelos** y aprende de las experiencias (requiere completar un episodio antes de calcular retornos)
- **DP:** Requiere modelo y utiliza **bootstraps** (actualiza $v(s)$ usando $v(s')$ de los estados vecinos)



Repaso: TD methods (learning states values)

Temporal-differences nos permite estimar la **función de valor** por medio del algoritmo:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) \right],$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t$$

- En cada paso t requiere $\{(s_t, r_{t+1}, s_{t+1})\}$
- Solamente estima la función de valor para una política dada (resuelve Bellman expectation equation)

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

- No estima el valor de las acciones ni calcula políticas óptimas

TD learning:

$$\left\{ \begin{array}{l} \overbrace{v_{t+1}(s_t)}^{\text{nueva est.}} = \overbrace{v_t(s_t)}^{\text{est. previa}} - \alpha(s_t) \underbrace{\left[v_t(s_t) - \overbrace{(r_{t+1} + \gamma v_t(s_{t+1}))}^{\text{TD target}} \right]}_{\text{TD error}}, \\ v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t \end{array} \right.$$

- **TD target:** el algoritmo lleva $v_t(s_t)$ hacia el TD target $\bar{v}_t := r_{t+1} + \gamma v_t(s_{t+1})$
- **TD error:** $\delta_t = v_t(s_t) - \bar{v}_t$ refleja la discrepancia entre los pasos de tiempo t y $t + 1$

Repaso: TD methods (learning action values)

SARSA nos permite estimar el **valor de la acción** por medio del algoritmo de TD:

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[\overbrace{q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))}^{\text{TD error}} \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

TD target

donde $t = 0, 1, 2, \dots$

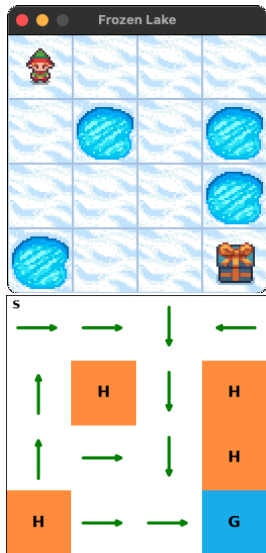
- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}$
- Estima el valor de la acción para una política dada al resolver la siguiente ecuación de Bellman:

→ $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$

- Junto a un paso de mejora de política **puede calcular políticas óptimas**

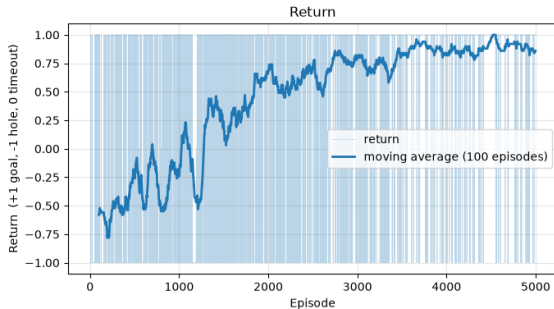
	TD learning	MC learning
Incremental	Sí. TD puede actualizar valores de estado/acción inmediatamente después de recibir una experiencia (online).	No. MC requiere que el episodio haya finalizado para calcular los retornos (offline).
Tareas	Tareas continuas y episódicas.	Solo tareas episódicas.
Bootstrapping	Sí. TD actualiza valores a partir de estimaciones previas.	No. Estima valores calculando retornos.
Sesgo y varianza	High bias / low estimation variance	Unbiased / high estimation variance

Ejemplos de SARSA

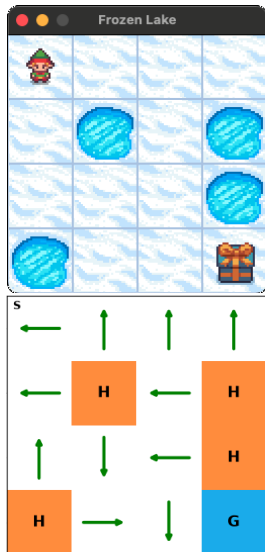


(Caso determinista)

```
# ----- #  
# Environment #  
# ----- #  
env = gym.make("FrozenLake-v1",  
               map_name="4x4",  
               is_slippery=False,  
               reward_schedule=(1, -1, 0), # (goal, hole, step)  
               render_mode="ansi")
```

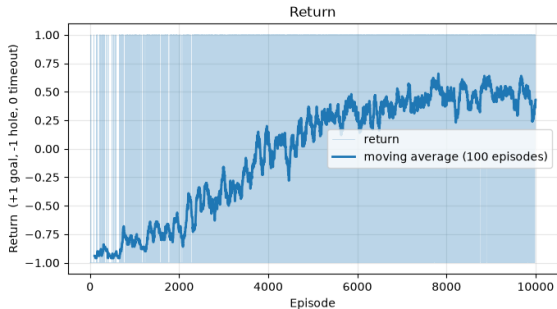


Ejemplos de SARSA



(Caso estocástico)

```
# ----- #  
# Environment #  
# ----- #  
env = gym.make("FrozenLake-v1",  
               map_name="4x4",  
               is_slippery=True,  
               reward_schedule=(1, -1, 0), # (goal, hole, step)  
               render_mode="ansi")
```



TD methods: Learning action values (n -step SARSA)

SARSA resuelve la siguiente ecuación de Bellman:

$$q_{\pi}(s, a) = \mathbb{E}[\underbrace{R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1})}_{G_t = R_{t+1} + \gamma G_{t+1}} | S_t = s, A_t = a]$$

$$\mathbb{E}(\underbrace{R_{t+1} + \gamma G_{t+1}})$$

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 G_{t+2}$$

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 G_{t+3}$$

$$\vdots$$

$$R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{u-1} R_{t+u} + \gamma^u G_{t+u}$$

TD methods: Learning action values (n -step SARSA)

SARSA resuelve la siguiente ecuación de Bellman:

$$\times \quad q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

n -step SARSA resuelve:

$$\times \quad q_{\pi}(s, a) = \mathbb{E}[\underbrace{R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n}} + \gamma^n q_{\pi}(S_{t+n}, A_{t+n}) | S_t = s, A_t = a]$$

A través del método iterativo: *TD learning*

$$\times \quad \begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(s_t, a_t) - \underbrace{(r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n}))}_{\text{TD target}} \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

“ n -step methods [...] free you from the tyranny of the time step”

TD methods: Learning action values (n -step SARSA)

SARSA:

necesita: $(\underline{s_t}, \underline{a_t}, \underline{r_{t+1}}, \underline{s_{t+1}}, \underline{a_{t+1}})$

para calcular:

$$\star \begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

n -step SARSA:

necesita: $(\underline{s_t}, \underline{a_t}, \underline{r_{t+1}}, \underline{s_{t+1}}, \underline{a_{t+1}}, \dots, \underline{r_{t+n}}, \underline{s_{t+n}}, \underline{a_{t+n}})$

para calcular:

$$\star \begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(s_t, a_t) - (\underline{r_{t+1}} + \gamma \underline{r_{t+2}} + \dots + \gamma^n q_t(\underline{s_{t+n}}, \underline{a_{t+n}})) \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

TD methods: Learning action values (variaciones de SARSA)

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

* Sarsa $\leftarrow G_t^{(1)} = \underline{R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1})}$
 $G_t^{(2)} = \underline{R_{t+1}} + \gamma \underline{R_{t+2}} + \gamma^2 \underline{q_{\pi}(S_{t+2}, A_{t+2})}$
 $\vdots$

n-step Sarsa $\leftarrow G_t^{(n)} = \underline{R_{t+1}} + \dots + \gamma \underline{R_{t+n}} + \gamma^n \underline{q_{\pi}(S_{t+n}, A_{t+n})}$
 $\vdots$

MC $\leftarrow G_t^{(\infty)} = \underline{R_{t+1}} + \gamma \underline{R_{t+2}} + \gamma^2 \underline{R_{t+3}} + \gamma^3 \underline{R_{t+4}} + \dots$

Existen otras variaciones como **Expected Sarsa** que resuelve la siguiente ecuación:

$$q_{\pi}(s, a) = \mathbb{E}\left[\underline{R_{t+1} + \gamma \mathbb{E}[q_{\pi}(S_{t+1}, A_{t+1} | S_{t+1} = s')]} \mid S_t = s, A_t = a\right]$$

see Section 6.6, Sutton & Barto, Reinforcement Learning

TD methods: Learning optimal action values (Q-learning)

- Q-learning fue desarrollado por Christopher Watkins en 1989 como parte de su trabajo doctoral en King's College.
- El algoritmo Q-learning es uno de los más clásicos en reinforcement learning.
- SARSA estima valores de acciones para una política dada, por lo que necesita de un paso de mejora de políticas para encontrar políticas óptimas.
- Por el contrario, Q-learning busca estimar acciones y políticas óptimas!



“I connected RL to Markov decision processes and invented Q-learning in the late 1980s in my unpublished PhD thesis”

Image from <https://www.cs.rhul.ac.uk/~chrisw/>

TD methods: Learning optimal action values (Q-learning)

En vez de resolver la ecuación de Bellman:

$$\star \quad q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Resolveremos la BOE en términos del valor de la acción:

$$\star \quad q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]$$

El algoritmo de Q-learning viene dado por:

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(s_t, a_t) - \underbrace{(r_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a'))}_{\text{TD target}} \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

TD learning

$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$

Algunos conceptos importantes:

- **Behavior policy:** política usada para generar las muestras/experiencias (política exploratoria).
- **Target policy:** política que se actualiza constantemente para converger a una política óptima (política de explotación).
- **On-policy learning:** cuando la behavior policy es la misma que la target policy.
- **Off-policy learning:** behavior policy y target policy son distintas.

SARSA es on-policy

- Resuelve la ecuación de Bellman para una política π dada:

$$\cancel{q_\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a],$$

donde $R_{t+1} \sim p(r|s, a)$, $S_{t+1} \sim p(s'|s, a)$ y $A_{t+1} \sim \pi(\cdot | S_{t+1})$.

- El algoritmo viene dado por:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(\underline{s_t}, \underline{a_t}) - (\underline{r_{t+1}} + \gamma q_t(\underline{s_{t+1}}, \underline{a_{t+1}})) \right]$$

- Dados (s_t, a_t) , r_{t+1} y s_{t+1} no dependen de la política.
- a_{t+1} se genera siguiendo $\pi(\cdot | s_{t+1})$.
- π_t genera episodios (behavior policy) y es la política que se mejora (target policy).

Q-learning es off-policy

- Resuelve la ecuación optimal de Bellman:

$$q(s, a) = \mathbb{E}[R_{t+1} + \gamma q \max_{a'}(S_{t+1}, a') | S_t = s, A_t = a],$$

donde $R_{t+1} \sim p(r|s, a)$, $S_{t+1} \sim p(s'|s, a)$ y ~~$A_{t+1} \sim \pi(\cdot | S_{t+1})$~~ .

- El algoritmo viene dado por:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(\underline{s_t}, \underline{a_t}) - (\underline{r_{t+1}} + \gamma \max_{a'} q_t(\underline{s_{t+1}}, a')) \right]$$

- Dados (s_t, a_t) , r_{t+1} y s_{t+1} no dependen de la política.
- No a_{t+1} es generado siguiendo $\pi(\cdot | s_{t+1})$. ✗
- La behavior policy solo genera a_t a partir de s_t y puede ser cualquiera (exploratoria). La target policy es la que se mejora para ser óptima.

TD methods: Learning optimal action values (Q-learning)

Algorithm 7.3: Optimal policy learning via Q-learning (off-policy version)

Initialization: Initial guess $q_0(s, a)$ for all (s, a) . Behavior policy $\pi_b(a|s)$ for all (s, a) . $\alpha_t(s, a) = \alpha > 0$ for all (s, a) and all t .

Goal: Learn an optimal target policy π_T for all states from the experience samples generated by π_b .

For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ generated by π_b , do

For each step $t = 0, 1, 2, \dots$ of the episode, do

Update q -value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$$

Update target policy for s_t :

$$\begin{cases} \pi_{T,t+1}(a|s_t) = 1 & \text{if } a = \arg \max_a q_{t+1}(s_t, a) \\ \pi_{T,t+1}(a|s_t) = 0 & \text{otherwise} \end{cases} \quad (\text{greedy})$$

* (revisar la versión on-policy también)

→ ϵ -greedy es caso on-policy

Los métodos presentados se pueden escribir de forma general como:

$$\text{TD} \quad * \quad q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) [q_t(s_t, a_t) - \bar{q}_t]$$

donde,

Método	TD target
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ ✓
n -step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n})$ ✓
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$ ✓
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$ ✓

Obs: En MC necesitamos definir $\alpha(s_t, a_t) = 1$

$$\text{MC} \quad q(s, a) = \frac{1}{N} \sum_{i=1}^N f^{(i)}(s, a)$$

Todos estos algoritmos son métodos de **aproximación estocástica** para algún problema en particular (Bellman o Bellman optimality):

Método	Problema que resuelven
Sarsa	$q(s, a) = \mathbb{E} [R_{t+1} + \gamma q(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$
n -step Sarsa	$q(s, a) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a]$
Q-learning	$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') \mid S_t = s, A_t = a \right]$
Monte Carlo	$q(s, a) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s, A_t = a]$

TD methods: Learning optimal action values (Q-learning)

Calcular una política óptima π^* usando Q-learning,

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a')) \right]$$

Ver código

`Example_01_optimal_policy_lesson_10_Q-learning.ipynb`
y completar la función `q_learning` y ajustar los parámetros de la función para obtener la política óptima.

recordar que:

- Entrar a una celda blanca, $r = 0$.
- Entrar a una celda naranja, $r = -1$.
- Alcanzar la celda azul, $r = +1$.

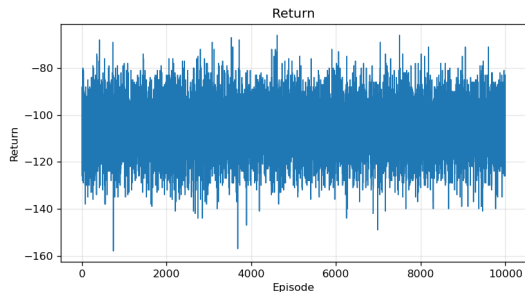
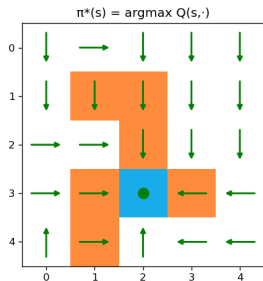
s_1	s_2	s_3	s_4	s_5
s_6	s_7	s_8	s_9	s_{10}
s_{11}	s_{12}	s_{13}	s_{14}	s_{15}
s_{16}	s_{17}	s_{18}	s_{19}	s_{20}
s_{21}	s_{22}	s_{23}	s_{24}	s_{25}

Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

TD methods: Learning optimal action values (Q-learning)

Parámetros utilizados: episodes = 10000, max episode length = 300, $\alpha_0 = 0.5$, $\alpha_{min} = 0.01$, decay every 500 episodes, decay rate = 0.9, and seed = 42,

$$V_{\text{Q-learning}} = \begin{bmatrix} 5.8 & 5.6 & 6.2 & 6.5 & 5.8 \\ 6.5 & 7.2 & 8.0 & 7.2 & 6.5 \\ 7.2 & 8.0 & 10.0 & 8.0 & 7.2 \\ 8.0 & 10.0 & 10.0 & 10.0 & 8.0 \\ 7.2 & 9.0 & 10.0 & 9.0 & 8.1 \end{bmatrix}, \quad V^* = \begin{bmatrix} 5.8 & 5.6 & 6.2 & 6.5 & 5.8 \\ 6.5 & 7.2 & 8.0 & 7.2 & 6.5 \\ 7.2 & 8.0 & 10.0 & 8.0 & 7.2 \\ 8.0 & 10.0 & 10.0 & 10.0 & 8.0 \\ 7.2 & 9.0 & 10.0 & 9.0 & 8.1 \end{bmatrix}$$



TD methods: Learning optimal action values (Q-learning)

Más ejemplos:

- Ver código `Example_02_optimal_policy_Q-learning_FrozenLake` y completar la función `q_learning` para obtener la política óptima.
- Ver código `Example_03_optimal_policy_Q-learning-Taxi.ipynb` y completar la función `q_learning` para obtener la política óptima.